Week 6 Assignment

Spark Architecture, Lazy Evaluation, Data Processing and File Format Optimization using Apache Spark

Objective - The objective of this assignment is to understand the architecture of Apache Spark and its execution model while performing efficient data processing using Spark Data Frames. The assignment covers Spark Architecture, Lazy Evaluation, DAG (Lineage Graph), schema handling, filtering, transformations, actions, file format optimization (CSV and Parquet), null handling, and building an end-to-end data processing pipeline.

Q1: Explain the roles of the **Driver**, **Cluster Manager**, and **Executor** in a Spark application.
Answer –
Driver – The Driver is the main process of a Spark application. It is responsible for creating the SparkSession, converting user code into execution plans (DAG), scheduling tasks, and coordinating the overall execution. It also collects the final results from the executors.

Cluster Manager - The Cluster Manager is responsible for managing the resources of the cluster. It allocates CPU cores and memory to Spark applications and launches executors on worker nodes. Spark can work with different cluster managers such as Standalone, YARN, Mesos, and Kubernetes.

Executor - Executors are worker processes that run on the worker nodes. They execute the tasks assigned by the Driver, perform computations, store intermediate data in memory or disk, and return the results back to the Driver.

Working Flow

1. The Driver receives the Spark application.
2. The Cluster Manager allocates the required resources.
3. Executors are launched on worker nodes.
4. Executors process the assigned tasks.
5. Results are sent back to the Driver.

Q2: How does Spark's **Lazy Evaluation** strategy improve performance when chain-processing large datasets?
Answer –
Lazy Evaluation - Lazy Evaluation is a core optimization feature of Apache Spark. When transformations such as filter(), select(), or withColumn() are applied, Spark does not execute them immediately. Instead, it records these operations in a **Directed Acyclic Graph (DAG)**, also known as the **Lineage Graph**. Execution begins only when an **Action** such as show(), count(), collect(), or write() is called.

How It Improves Performance

- Spark combines multiple transformations into a single optimized execution plan.
- It avoids unnecessary intermediate computations.
- It minimizes disk I/O by processing data efficiently in memory.
- It optimizes task execution using the DAG Scheduler.
- It executes only the operations required to produce the final result.
-

Example

```
df.filter(df["price"] > 100) \
  .select("product_id", "price") \
  .show()
```

In this example, the filter() and select() transformations are **not executed immediately**. Spark records them in the DAG and performs both operations together only when show() is called.

Q3: Write a Spark command to read a CSV file located at `"data/source.csv"`, ensuring the first row is treated as a header and `inferSchema` is enabled.
Answer - The following Spark command reads a CSV file while treating the first row as the header and automatically inferring the data types of each column using inferSchema=True.

Colab Code -

```
df = spark.read.csv(
    "ecommerce_orders.csv",
    header=True,
    inferSchema=True
)

print("Dataset Loaded Successfully!\n")

df.show(5)
```

```
Dataset Loaded Successfully!

+-------+----------+-----------+--------+-------+----------+------+---------+------+--------+
|user_id|product_id|   category|old_name|  price|base_price|amount|   status|region|priority|
+-------+----------+-----------+--------+-------+----------+------+---------+------+--------+
|   U001|      P101|Electronics|  Item_1| 178.18|       151|534.54|Completed| South|    High|
|   U002|      P102|Electronics|  Item_2| 1752.3|      1485|8761.5|Completed|  West|    High|
|   U003|      P103|Electronics|  Item_3| 343.38|       291|686.76|Completed| North|     Low|
|   U004|      P104|   Clothing|  Item_4|1847.88|      1566|9239.4|  Pending| South|  Medium|
|   U005|      P105|     Sports|  Item_5| 789.42|       669|789.42|Completed|  West|  Medium|
+-------+----------+-----------+--------+-------+----------+------+---------+------+--------+
only showing top 5 rows
```

```
df.printSchema()
```

```
root
 |-- user_id: string (nullable = true)
 |-- product_id: string (nullable = true)
 |-- category: string (nullable = true)
 |-- old_name: string (nullable = true)
 |-- price: double (nullable = true)
 |-- base_price: integer (nullable = true)
 |-- amount: double (nullable = true)
 |-- status: string (nullable = true)
 |-- region: string (nullable = true)
 |-- priority: string (nullable = true)
```

Explanation

- spark.read.csv() reads the CSV file.
- header=True treats the first row as column names.
- inferSchema=True automatically detects the appropriate data type for each column.
- show(5) displays the first five records of the DataFrame.

Q4: What is the difference between **CSV** and **Parquet** in terms of storage (row-based vs. columnar) and why does it matter for performance?

Answer -

CSV (Row-Based Storage) - CSV is a row-based file format in which data is stored sequentially row by row. It is simple, human-readable, and widely supported, but it does not store metadata such as data types. As a result, Spark often needs to infer the schema, which increases processing time.

Parquet (Columnar Storage) - Parquet is a columnar storage format that stores data column by column instead of row by row. It supports schema information, efficient compression, and advanced optimizations such as Predicate Pushdown. Since only the required columns are read, Parquet significantly reduces disk I/O and improves query performance.

Difference Between CSV and Parquet

CSV	Parquet
Row-based storage	Column-based storage
Larger file size	Smaller due to compression
No schema metadata	Stores schema information
Reads entire rows	Reads only required columns
Slower for analytics	Faster for analytical workloads

Why It Matters for Performance

Parquet improves performance because Spark reads only the required columns instead of the entire dataset. Its columnar storage, compression, and built-in optimizations reduce memory usage and disk access, making it much faster than CSV for large-scale data processing.

Q5: Given a DataFrame `df`, write a query to select the columns `product_id` and `price` where the `category` is 'Electronics'.
Answer - The following query filters the DataFrame to retrieve only the records where the **category** is **Electronics**. It then selects only the **product_id** and **price** columns.

Colab Code -

```
from pyspark.sql.functions import col
electronics_df = df.filter(
    col("category") == "Electronics"
).select(
    "product_id",
    "price"
)
electronics_df.show()
```

```
+----------+-------+
|product_id|  price|
+----------+-------+
|      P101| 178.18|
|      P102| 1752.3|
|      P103| 343.38|
|      P110| 227.74|
|      P111|1036.04|
|      P113|1589.46|
|      P122|1047.84|
|      P137|2305.72|
|      P142| 264.32|
+----------+-------+
```

Explanation

- filter() is used to select only the rows where the **category** is **Electronics**.
- select() retrieves only the **product_id** and **price** columns.
- show() displays the filtered results.

Q6: Write the code to "revise" a DataFrame by renaming the column `old_name` to `new_name` and casting the `price` column from a String to a Double.

Answer - The following code renames the **old_name** column to **new_name** and converts the **price** column from **String** to **Double** using the cast() function.

Colab Code -

```
from pyspark.sql.functions import col
from pyspark.sql.types import StringType, DoubleType

df_string = df.withColumn(
    "price",
    col("price").cast(StringType())
)

updated_df = df_string.withColumnRenamed(
    "old_name",
    "new_name"
).withColumn(
    "price",
    col("price").cast(DoubleType())
)

updated_df.printSchema()
updated_df.show(5)
```

```
root
 |-- user_id: string (nullable = true)
 |-- product_id: string (nullable = true)
 |-- category: string (nullable = true)
 |-- new_name: string (nullable = true)
 |-- price: double (nullable = true)
 |-- base_price: integer (nullable = true)
 |-- amount: double (nullable = true)
 |-- status: string (nullable = true)
 |-- region: string (nullable = true)
 |-- priority: string (nullable = true)

+-------+----------+-----------+--------+-------+----------+------+---------+------+--------+
|user_id|product_id|   category|new_name|  price|base_price|amount|   status|region|priority|
+-------+----------+-----------+--------+-------+----------+------+---------+------+--------+
|   U001|      P101|Electronics|  Item_1| 178.18|       151|534.54|Completed| South|    High|
|   U002|      P102|Electronics|  Item_2| 1752.3|      1485|8761.5|Completed|  West|    High|
|   U003|      P103|Electronics|  Item_3| 343.38|       291|686.76|Completed| North|     Low|
|   U004|      P104|   Clothing|  Item_4|1847.88|      1566|9239.4|  Pending| South|  Medium|
|   U005|      P105|     Sports|  Item_5| 789.42|       669|789.42|Completed|  West|  Medium|
+-------+----------+-----------+--------+-------+----------+------+---------+------+--------+
only showing top 5 rows
```

Explanation

- withColumnRenamed() renames the **old_name** column to **new_name**.
- cast(StringType()) converts the **price** column to String for demonstration.
- cast(DoubleType()) converts the **price** column back to Double.
- printSchema() verifies that the column type has been updated successfully.

Q7: How does Spark use the **Lineage Graph (DAG)** to provide fault tolerance if a worker node fails?

Answer -

Lineage Graph (DAG) - The Lineage Graph, also known as the Directed Acyclic Graph (DAG), records the sequence of transformations applied to a DataFrame or RDD. Instead of storing multiple copies of intermediate data, Spark keeps track of how each dataset was created.

Fault Tolerance Using DAG - If a worker node or executor fails, Spark does not need to restart the entire application. Using the Lineage Graph, Spark identifies the lost partition and recomputes only the missing data by re-executing the required transformations. This recovery process is automatic and ensures reliable execution without manual intervention.

Advantages

- Automatically recovers lost data partitions.
- Recomputes only the affected data instead of the entire dataset.
- Eliminates the need for data replication in most cases.
- Improves reliability while reducing storage overhead.
- Ensures efficient fault recovery in distributed environments.

Q8: Write a query to filter a DataFrame `df_orders` for rows where the `status` is 'Completed' AND the `amount` is greater than 1000.

Answer - The following query filters the DataFrame to retrieve only those records where the **status** is **Completed** and the **amount** is greater than **1000**.

Colab Code -

```
from pyspark.sql.functions import col

completed_orders = df.filter(
    (col("status") == "Completed") &
    (col("amount") > 1000)
)
completed_orders.show()
```

```
+-------+----------+-----------+--------+-------+----------+-------+---------+------+--------+
|user_id|product_id|   category|old_name|  price|base_price| amount|   status|region|priority|
+-------+----------+-----------+--------+-------+----------+-------+---------+------+--------+
|   U002|      P102|Electronics|  Item_2| 1752.3|      1485| 8761.5|Completed|  West|    High|
|   U015|      P115|    Grocery| Item_15|2154.68|      1826|2154.68|Completed| North|  Medium|
|   U017|      P117|  Furniture| Item_17| 1073.8|       910| 4295.2|Completed|  East|    High|
|   U020|      P120|   Clothing| Item_20|1348.74|      1143|5394.96|Completed| North|    High|
|   U029|      P129|    Grocery| Item_29| 1298.0|      1100| 1298.0|Completed|  East|  Medium|
|   U038|      P138|   Clothing| Item_38|  790.6|       670| 3162.4|Completed| South|     Low|
|   U039|      P139|     Sports| Item_39| 1510.4|      1280| 6041.6|Completed|  West|  Medium|
|   U045|      P145|  Furniture| Item_45|2069.72|      1754|10348.6|Completed| North|     Low|
|   U046|      P146|     Sports| Item_46|2138.16|      1812|2138.16|Completed| South|    High|
|   U047|      P147|  Furniture| Item_47|1290.92|      1094|5163.68|Completed|  West|    High|
|   U050|      P150|   Clothing| Item_50| 834.26|       707|1668.52|Completed| North|     Low|
+-------+----------+-----------+--------+-------+----------+-------+---------+------+--------+
```

Explanation

- filter() is used to apply conditions on the DataFrame.
- col("status") == "Completed" selects only completed orders.
- col("amount") > 1000 filters orders with an amount greater than 1000.
- The & operator ensures that **both conditions** must be satisfied.
- show() displays the filtered records.

Q9: Explain the concept of **Predicate Pushdown** in Parquet and how it affects the amount of data loaded into memory.

Answer –

Predicate Pushdown - Predicate Pushdown is an optimization technique used by Spark when reading Parquet files. Instead of loading the entire dataset into memory and then applying filter conditions, Spark pushes the filter operation down to the Parquet storage layer.

As a result, only the rows that satisfy the filter condition are read from disk, while the remaining data is skipped.

How It Improves Performance

- Reduces the amount of data read from disk.
- Loads only the required records into memory.
- Decreases disk I/O operations.
- Reduces memory consumption.
- Improves query execution speed, especially for large datasets.

Example

If a Parquet file contains millions of records and the query filters for:

df.filter(df["category"] == "Electronics")

Spark reads only the data blocks containing the required records instead of scanning the entire dataset.

Q10: Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).
Answer - The following code creates a new column named **final_price** by multiplying the **base_price** column by **1.18**, representing an 18% tax.

Colab Code -

```
from pyspark.sql.functions import col

final_price_df = df.withColumn(
    "final_price",
    col("base_price") * 1.18
)
final_price_df.show(5)
```

```
+-------+----------+-----------+--------+-------+----------+------+---------+------+--------+------------------+
|user_id|product_id|   category|old_name|  price|base_price|amount|   status|region|priority|       final_price|
+-------+----------+-----------+--------+-------+----------+------+---------+------+--------+------------------+
|   U001|      P101|Electronics|  Item_1| 178.18|       151|534.54|Completed| South|    High|178.17999999999998|
|   U002|      P102|Electronics|  Item_2| 1752.3|      1485|8761.5|Completed|  West|    High|            1752.3|
|   U003|      P103|Electronics|  Item_3| 343.38|       291|686.76|Completed| North|     Low|            343.38|
|   U004|      P104|   Clothing|  Item_4|1847.88|      1566|9239.4|  Pending| South|  Medium|1847.8799999999999|
|   U005|      P105|     Sports|  Item_5| 789.42|       669|789.42|Completed|  West|  Medium|            789.42|
+-------+----------+-----------+--------+-------+----------+------+---------+------+--------+------------------+
only showing top 5 rows
```

Explanation

- withColumn() is used to create a new column in the DataFrame.
- col("base_price") * 1.18 calculates the final price after adding an 18% tax.
- The newly created column is named **final_price**.
- show(5) displays the first five records with the new column.

Q11: What is the difference between **Transformations** and **Actions**? Provide two examples of each.
Answer -
Transformations - Transformations are operations that create a new DataFrame or RDD from an existing one without executing immediately. They are **lazy**, meaning Spark only records them in the **Lineage Graph (DAG)** and waits until an Action is called before executing them.
Examples of Transformations:

- filter()
- select()

Actions - Actions are operations that trigger the execution of all pending transformations. They produce a result, either by displaying data, returning a value to the Driver, or writing data to storage.
Examples of Actions:

- show()
- count()

Difference Between Transformations and Actions

Transformations	Actions
Lazy in nature	Trigger execution
Build the DAG	Execute the DAG
Return a new DataFrame/RDD	Return a result or write output
Do not perform computation immediately	Perform the actual computation

Example

```
filtered_df = df.filter(df["price"] > 500)
selected_df = filtered_df.select("product_id", "price")
selected_df.show()
```

In the above example, filter() and select() are **Transformations**, while show() is an **Action** that triggers the execution of the entire pipeline.

Q12: Write the Spark command to load a Parquet file from `"path/to/input"`, filter out any rows where `user_id` is null, and save the result as a CSV at `"path/to/output"`.

Answer - The following code first saves the existing DataFrame as a **Parquet** file. It then loads the Parquet file, filters out rows where the **user_id** is null, and finally saves the filtered data as a **CSV** file.

Colab Code -

Step 1: Save CSV DataFrame as Parquet

```
df.write.mode("overwrite").parquet("input_parquet")
```

Step 2: Read the Parquet File

```
parquet_df = spark.read.parquet("input_parquet")

parquet_df.show(5)
```

```
+-------+----------+-----------+--------+-------+----------+------+---------+------+--------+
|user_id|product_id|   category|old_name|  price|base_price|amount|   status|region|priority|
+-------+----------+-----------+--------+-------+----------+------+---------+------+--------+
|   U001|      P101|Electronics|  Item_1| 178.18|       151|534.54|Completed| South|    High|
|   U002|      P102|Electronics|  Item_2| 1752.3|      1485|8761.5|Completed|  West|    High|
|   U003|      P103|Electronics|  Item_3| 343.38|       291|686.76|Completed| North|     Low|
|   U004|      P104|   Clothing|  Item_4|1847.88|      1566|9239.4|  Pending| South|  Medium|
|   U005|      P105|     Sports|  Item_5| 789.42|       669|789.42|Completed|  West|  Medium|
+-------+----------+-----------+--------+-------+----------+------+---------+------+--------+
only showing top 5 rows
```

Step 3: Filter Null user_id

```
from pyspark.sql.functions import col

filtered_df = parquet_df.filter(
    col("user_id").isNotNull()
)

filtered_df.show(5)
```

```
+-------+----------+-----------+--------+-------+----------+------+---------+------+--------+
|user_id|product_id|   category|old_name|  price|base_price|amount|   status|region|priority|
+-------+----------+-----------+--------+-------+----------+------+---------+------+--------+
|   U001|      P101|Electronics|  Item_1| 178.18|       151|534.54|Completed| South|    High|
|   U002|      P102|Electronics|  Item_2| 1752.3|      1485|8761.5|Completed|  West|    High|
|   U003|      P103|Electronics|  Item_3| 343.38|       291|686.76|Completed| North|     Low|
|   U004|      P104|   Clothing|  Item_4|1847.88|      1566|9239.4|  Pending| South|  Medium|
|   U005|      P105|     Sports|  Item_5| 789.42|       669|789.42|Completed|  West|  Medium|
+-------+----------+-----------+--------+-------+----------+------+---------+------+--------+
only showing top 5 rows
```

Step 4: Save as CSV

```
filtered_df.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("output_csv")
```

Step 5: Verify Output

```
import os

print(os.listdir("output_csv"))
```

```
['._SUCCESS.crc', 'part-00000-a2e00ee8-fc0e-4661-b2d0-d4c35fc5709e-c000.csv', '.part-00000-a2e00ee8-fc0e-4661-b2d0-d4c35fc5709e-c000.csv.crc']
```

Explanation

- parquet() saves the DataFrame in Parquet format.
- read.parquet() loads the Parquet file.
- isNotNull() removes records with null user_id.
- write.csv() saves the filtered data as a CSV file with column headers.
- mode("overwrite") replaces any existing output directory.

Q13: In Spark Architecture, what is the difference between **Client Mode** and **Cluster Mode**?

Answer -
Client Mode - In Client Mode, the **Driver program runs on the client machine** from which the Spark application is submitted. The Executors run on the worker nodes of the cluster. Since the Driver is outside the cluster, the client machine must remain active throughout the execution of the application.
Cluster Mode - In Cluster Mode, the **Driver program runs inside the cluster** on one of the worker nodes. The Cluster Manager is responsible for launching both the Driver and the Executors. The client can disconnect after submitting the application because the cluster manages the execution independently.

Difference Between Client Mode and Cluster Mode

Client Mode	**Cluster Mode**
Driver runs on the client machine	Driver runs inside the cluster
Client machine must remain active	Client can disconnect after job submission
Suitable for development and testing	Suitable for production environments
Dependent on client availability	Managed entirely by the cluster

Q14: Write a query to filter a dataset for rows where the `region` is 'North' OR the `priority` is 'High'.

Answer - The following query filters the DataFrame to retrieve all records where the **region** is **North** or the **priority** is **High**.

Colab Code -

```
from pyspark.sql.functions import col

filtered_region_priority = df.filter(
    (col("region") == "North") |
    (col("priority") == "High")
)

filtered_region_priority.show()
```

```
+-------+----------+-----------+--------+-------+----------+-------+---------+------+--------+
|user_id|product_id|   category|old_name|  price|base_price| amount|   status|region|priority|
+-------+----------+-----------+--------+-------+----------+-------+---------+------+--------+
|   U001|      P101|Electronics|  Item_1| 178.18|       151| 534.54|Completed| South|    High|
|   U002|      P102|Electronics|  Item_2| 1752.3|      1485| 8761.5|Completed|  West|    High|
|   U003|      P103|Electronics|  Item_3| 343.38|       291| 686.76|Completed| North|     Low|
|   U006|      P106|    Grocery|  Item_6| 493.24|       418| 986.48|  Pending| North|    High|
|   U007|      P107|  Furniture|  Item_7| 351.64|       298|1054.92|  Pending|  East|    High|
|   U008|      P108|  Furniture|  Item_8|1413.64|      1198|1413.64|  Pending| North|     Low|
|   NULL|      P110|Electronics| Item_10| 227.74|       193| 455.48|  Pending| North|    High|
|   U011|      P111|Electronics| Item_11|1036.04|       878|3108.12|  Pending|  East|    High|
|   U013|      P113|Electronics| Item_13|1589.46|      1347|3178.92|Cancelled| South|    High|
|   U015|      P115|    Grocery| Item_15|2154.68|      1826|2154.68|Completed| North|  Medium|
|   U016|      P116|  Furniture| Item_16| 764.64|       648| 764.64|Completed|  East|    High|
|   U017|      P117|  Furniture| Item_17| 1073.8|       910| 4295.2|Completed|  East|    High|
|   U019|      P119|     Sports| Item_19|1152.86|       977| 5764.3|  Pending|  East|    High|
|   U020|      P120|   Clothing| Item_20|1348.74|      1143|5394.96|Completed| North|    High|
|   U023|      P123|    Grocery| Item_23|1454.94|      1233|1454.94|Cancelled| North|     Low|
|   U024|      P124|     Sports| Item_24|1931.66|      1637|5794.98|Cancelled|  East|    High|
|   NULL|      P125|    Grocery| Item_25| 1168.2|       990| 2336.4|  Pending| North|     Low|
|   U026|      P126|    Grocery| Item_26| 1327.5|      1125| 2655.0|Cancelled| North|     Low|
|   U027|      P127|    Grocery| Item_27|2151.14|      1823|10755.7|Cancelled| South|    High|
|   U028|      P128|    Grocery| Item_28|1959.98|      1661|3919.96|Cancelled| North|     Low|
+-------+----------+-----------+--------+-------+----------+-------+---------+------+--------+
only showing top 20 rows
```

Explanation

- filter() is used to apply conditions to the DataFrame.
- col("region") == "North" selects records belonging to the North region.
- col("priority") == "High" selects records with High priority.
- The | (OR) operator ensures that records satisfying **either** condition are returned.
- show() displays the filtered results.

Q15: When exploring a dataset, why is it safer to use `.show(5)` instead of `.collect()` on a multi-terabyte dataset?

Answer –

.show(5) - The .show(5) function displays only the first five rows of a DataFrame. It retrieves a small subset of data for inspection, making it memory-efficient and suitable for exploring large datasets.

.collect() - The .collect() function retrieves **all the records** from the executors and transfers them to the Driver program as a local collection. For very large datasets, this can consume a significant amount of memory and may cause the Driver to crash with an **Out of Memory (OOM)** error.

Why .show(5) is Safer

- Retrieves only a few rows instead of the entire dataset.
- Consumes very little memory on the Driver.
- Prevents Out of Memory (OOM) errors.
- Faster for inspecting data.
- Recommended for exploring large or multi-terabyte datasets.

Difference Between .show(5) and .collect()

.show(5)	**.collect()**
Displays only the first five rows	Retrieves all rows
Memory efficient	Memory intensive
Suitable for large datasets	Suitable only for small datasets
Used for data exploration	Used when the complete dataset is required in the Driver